

- 人工智能实验报告 第7周
 - 一.实验题目：购房预测分类任务
 - 二.实验内容
 - 1.算法原理
 - 2.关键代码展示(可选)
 - 三.实验结果及分析
 - 1.实验结果展示示例（可截图可表可文字，尽量可视化）
 - 2.评测指标展示及分析（机器学习实验必须有此项，其它可分析运行时间等）

人工智能实验报告 第7周

姓名:陈安康 学号:22320021

一.实验题目：购房预测分类任务

二.实验内容

1.算法原理

通过用户年龄与用户薪资两个特征量，采用感知机算法与逻辑回归算法来进行预测用户是否购房。通过将两个特征量通过线性组合输出得到分数，再将分数输入逻辑函数再转换成分类标签（0代表不购房，1代表购房）进行预测。数学公式如下：

$$z = Sigmoid(y), \quad y = w_{age} * x_{age} + w_{salary} * x_{salary} + b$$

$$step(z) = \begin{cases} 0, & \text{if } z < 0.5 \\ 1, & \text{if } z \geq 0.5 \end{cases}$$

接下来关键的是对权重 w_{age} 、 w_{salary} 、 b 三个权重的训练，思路是给定训练集，并定义损失函数，通过梯度下降法最小化损失函数。损失函数采用交叉熵函数，由于只有两个特征量，模型简单，所以不需要添加正则项，假设训练集大小为 n ，交叉熵函数定义如下：

$$L_{CE} = - \sum_{i=1}^n y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})$$

其中 $\hat{y}^{(i)}$ 是第i个训练数据的预测值， $y^{(i)}$ 是第i个数据的真实值。

对于第i项数据的损失函数，其求偏导，应用链式法则得到某个方向上的梯度，其中j表示第j个权重：

$$\frac{\partial L_{CE}^{(i)}}{\partial w_j} = (\hat{y}^{(i)} - y^{(i)}) * x_j^{(i)}$$

由于数据量小，所以采用BGD算法即可。设用 α 表示学习率，则每一次迭代，更新权重的公式为：

$$w_j = w_j - \alpha * \frac{\sum_{i=1}^n \frac{\partial L_{CE}^{(i)}}{\partial w_j}}{n}$$

在进行如上步骤之前，还需要对数据进行归一化与标准化，使数据尺度一致，加速收敛。公式分别如下：

$$x_{\text{normalized}} = \frac{x - \min(x)}{\max(x) - \min(x)}$$

$$x_{\text{standardized}} = \frac{x - \mu}{\sigma}$$

选取给定的400项数据的一部分作为训练集进行训练，得到合适的权重之后，进行预测，得到决策边界图以及loss曲线图以及预测准确率。

代码实现上，通过一个类来进行实现，首先创建两个对象，一个用来训练得到合适权重，再将权重传递给另外一个对象进行预测。

2.关键代码展示(可选)

下图展示了关键的参数训练以及预测代码

```

# 训练参数用的，采用批量梯度下降
def train(self):
    for i in range(self.n):
        y = self.sigmoid(np.dot(self.data, self.param))
        self.loss(y)
        for j in range(3):
            grad = np.dot(self.data[:,j].T, (y - self.result))
            self.param[j][0] = self.param[j][0] - self.learn_rate * (1 / self.size) * grad

# 用来预测的
def predict(self):
    # self.predict = np.round(self.sigmoid(np.dot(self.data, self.param)))
    predict = np.round(self.sigmoid(np.dot(self.data, self.param)))
    count = 0
    for i in range(self.size):
        if predict[i][0] == self.result[i][0]:
            count += 1
    percent = float(count) / self.size
    return predict, percent

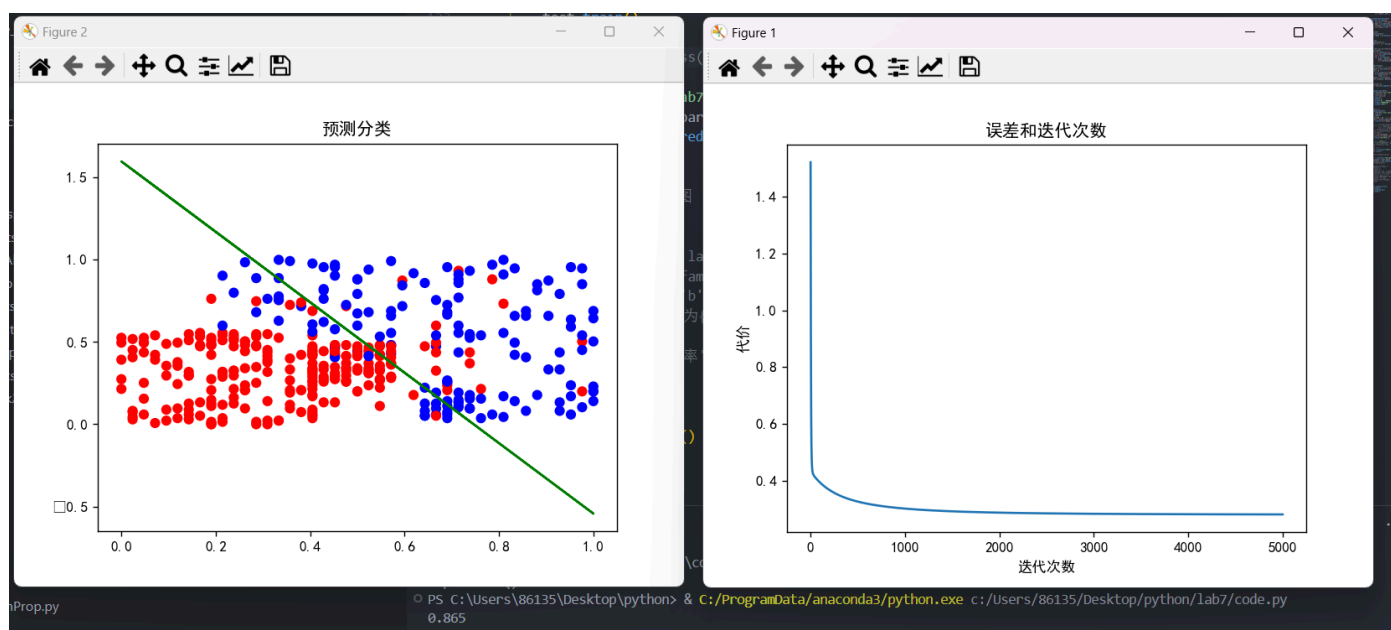
# 求损失函数平均值，绘图用的
def loss(self, y):
    self.loss_val.append(-np.mean(self.result * np.log(y) + (1 - self.result) * np.log(1 - y)))

```

三.实验结果及分析

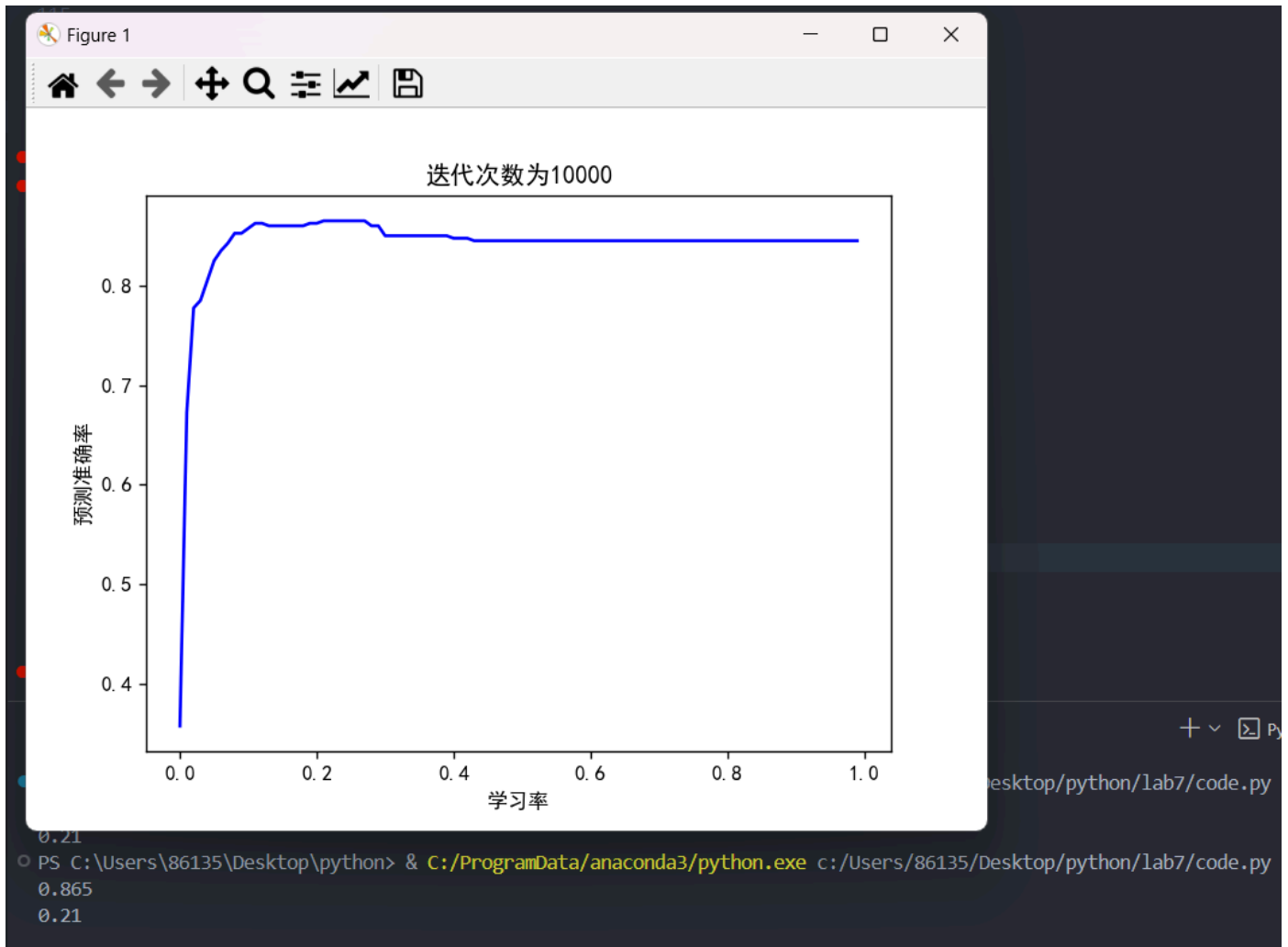
1.实验结果展示示例（可截图可表可文字，尽量可视化）

在采用训练集大小为140，迭代次数为5000，学习率为0.46时，预测准确率为86.5%(在命令行中打印)。结果如下：

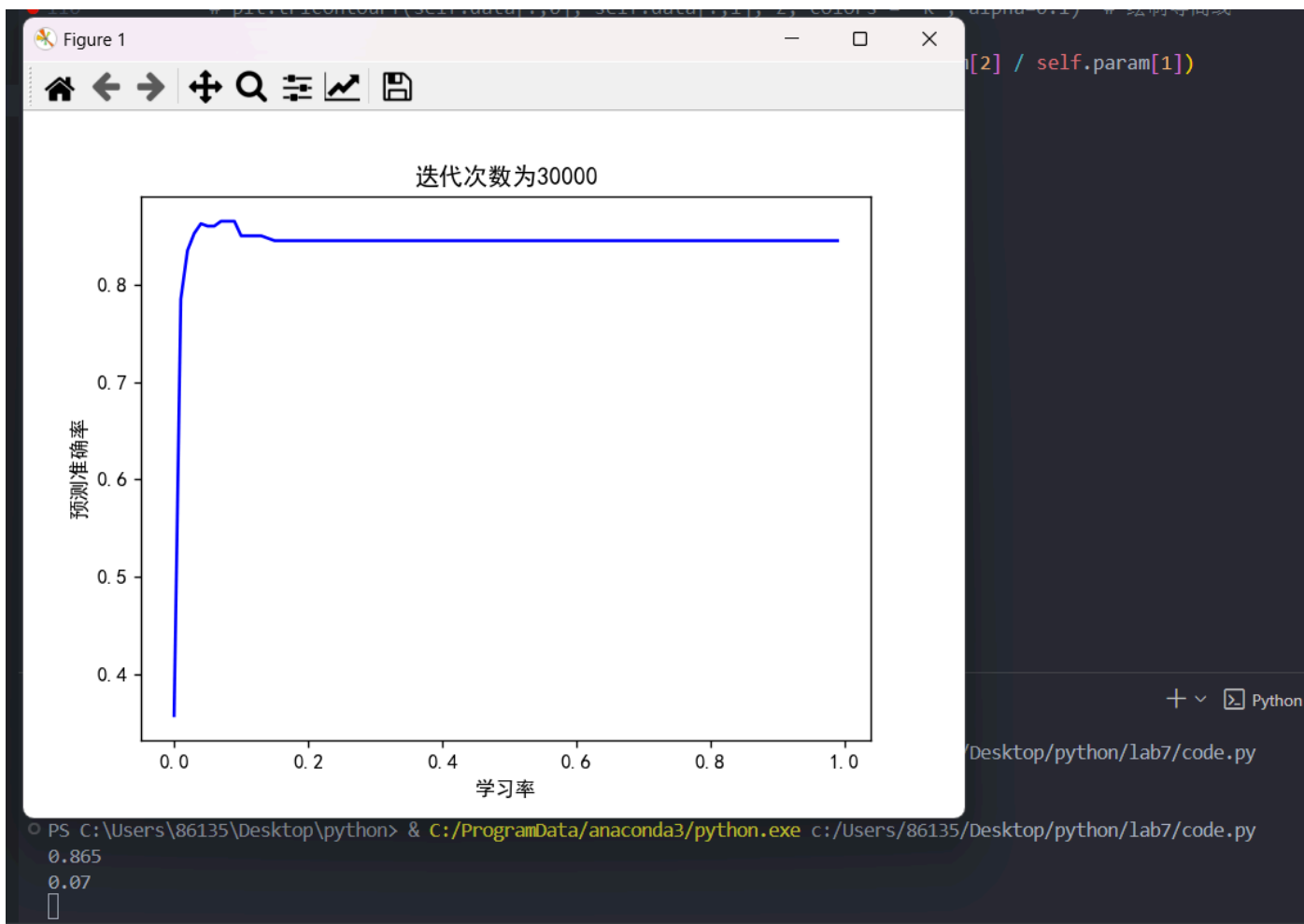
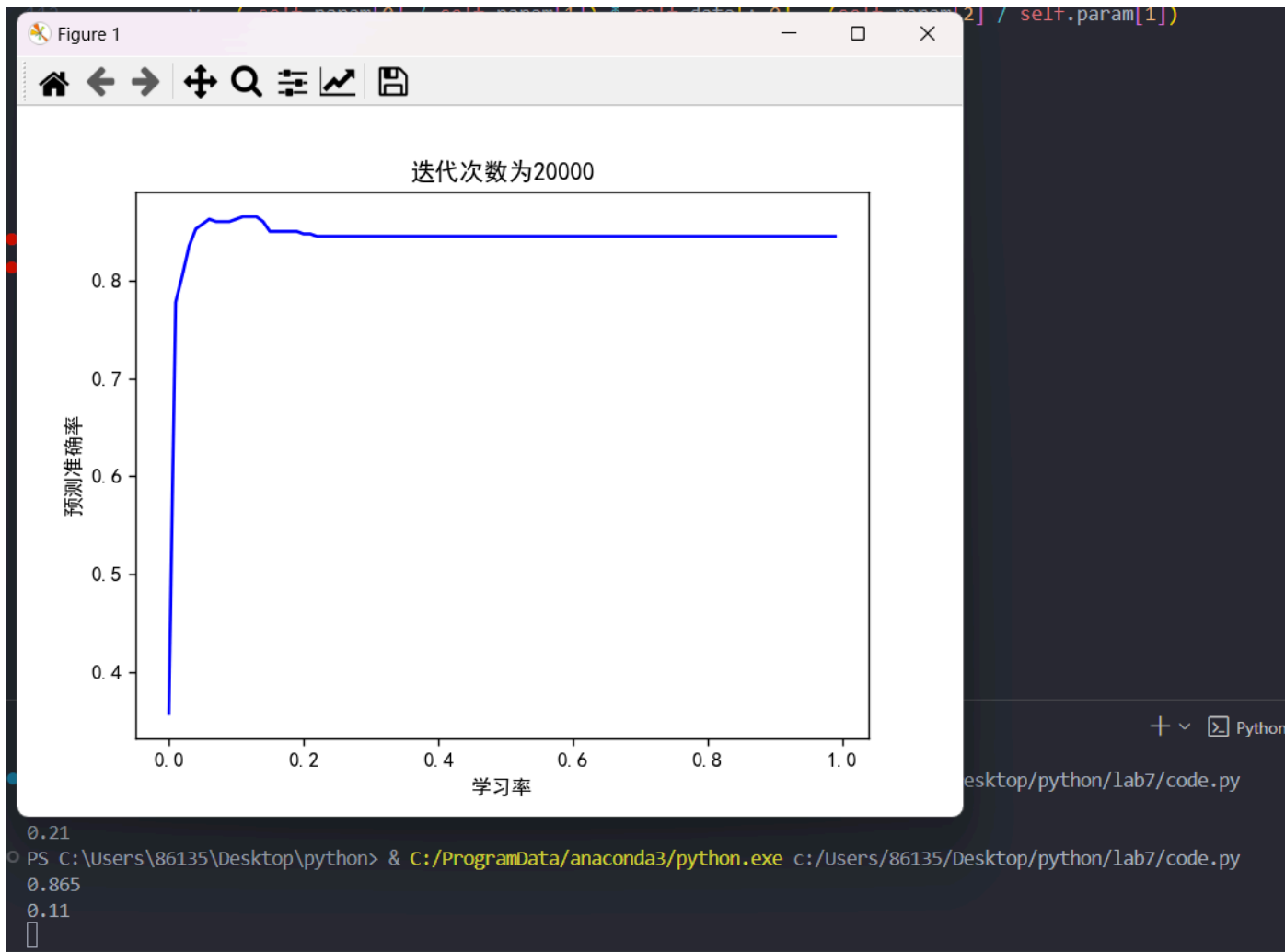


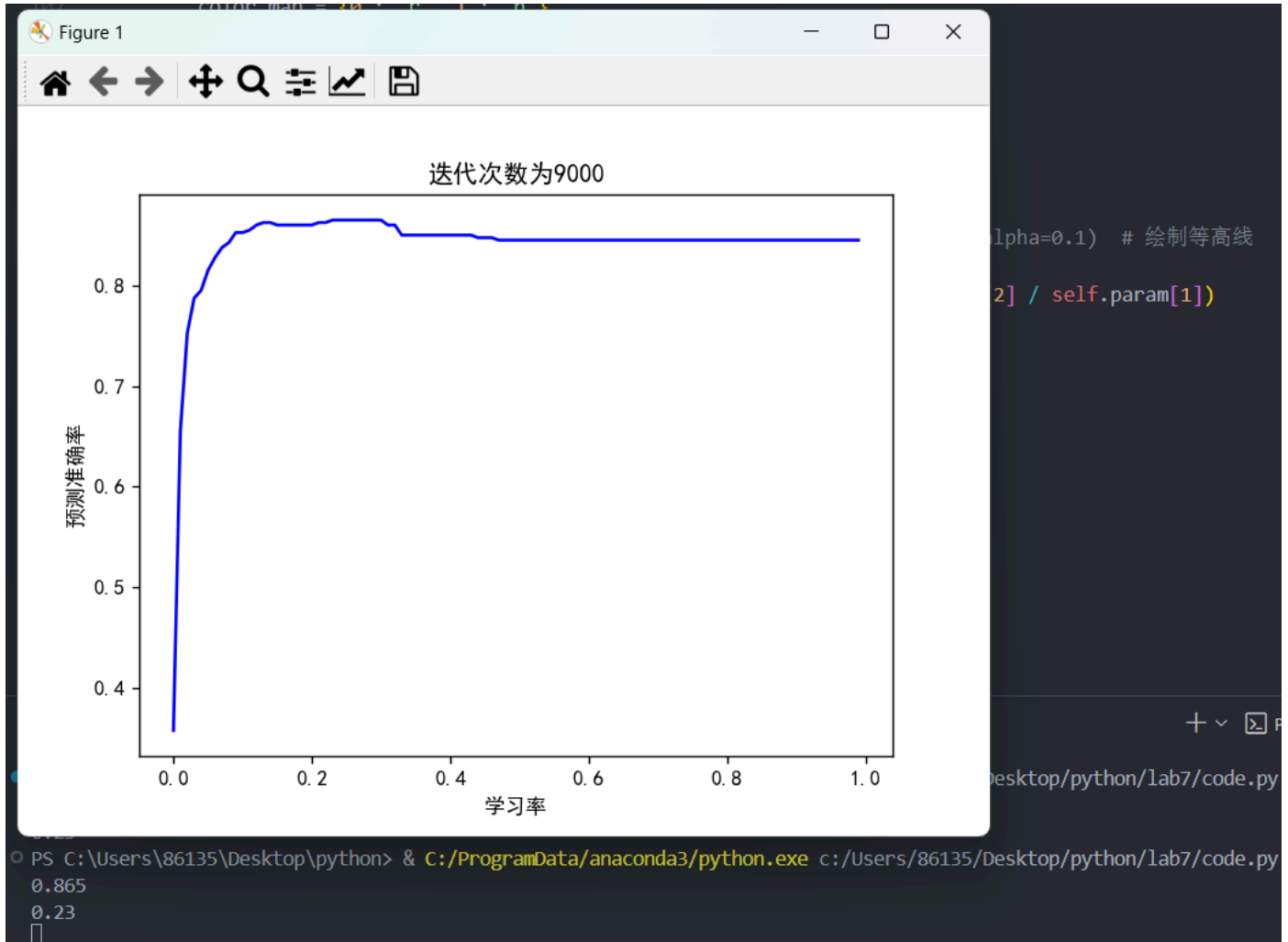
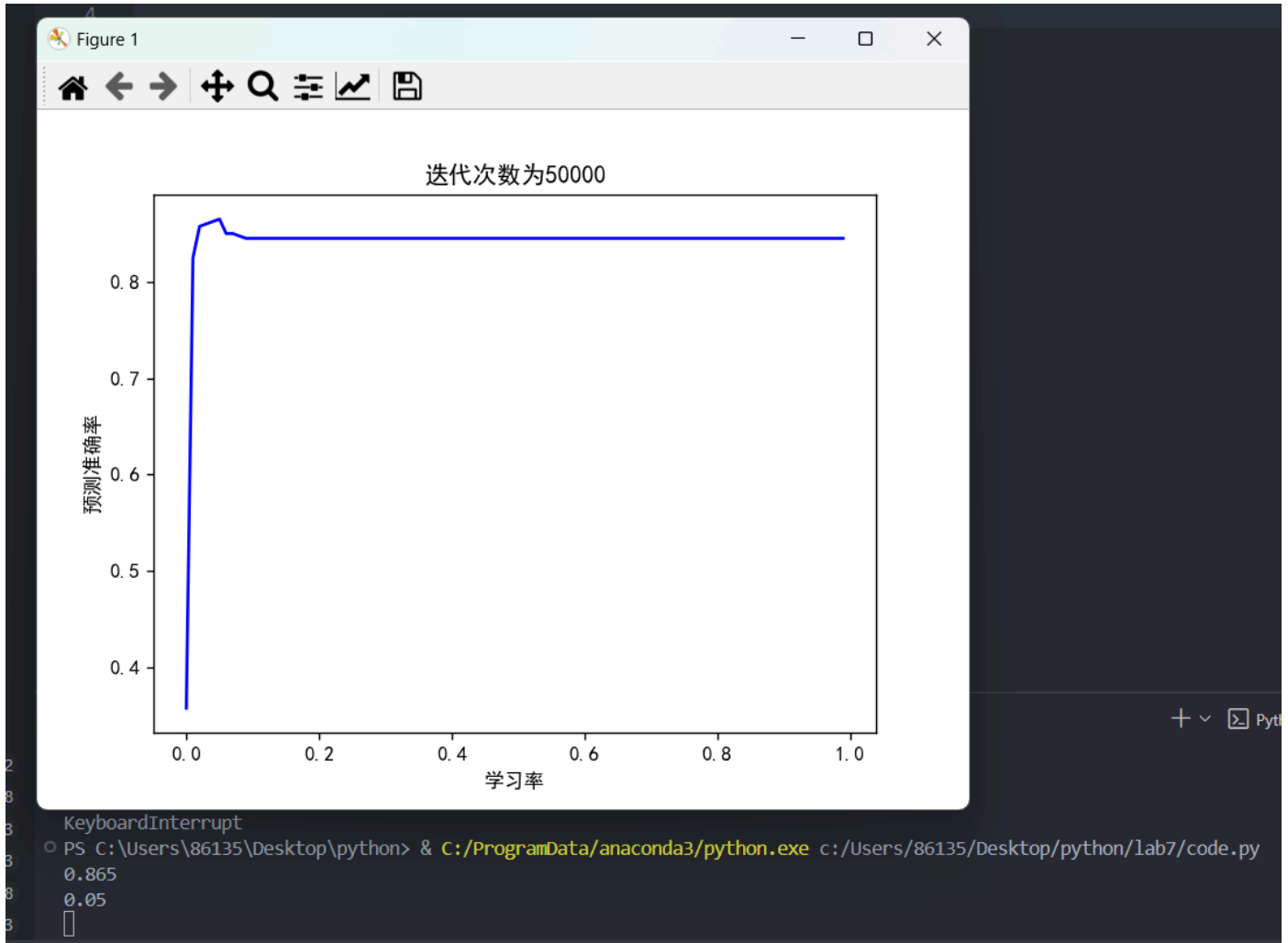
上述结果为该训练集约束下所能达到的最好效果，理由如下：

在固定训练集下，修改类，将学习率修改为可以传入的参数，接着在不同迭代次数下，遍历学习率的取值，进行训练与预测，期望找到最优的学习率，得到结果图如下：

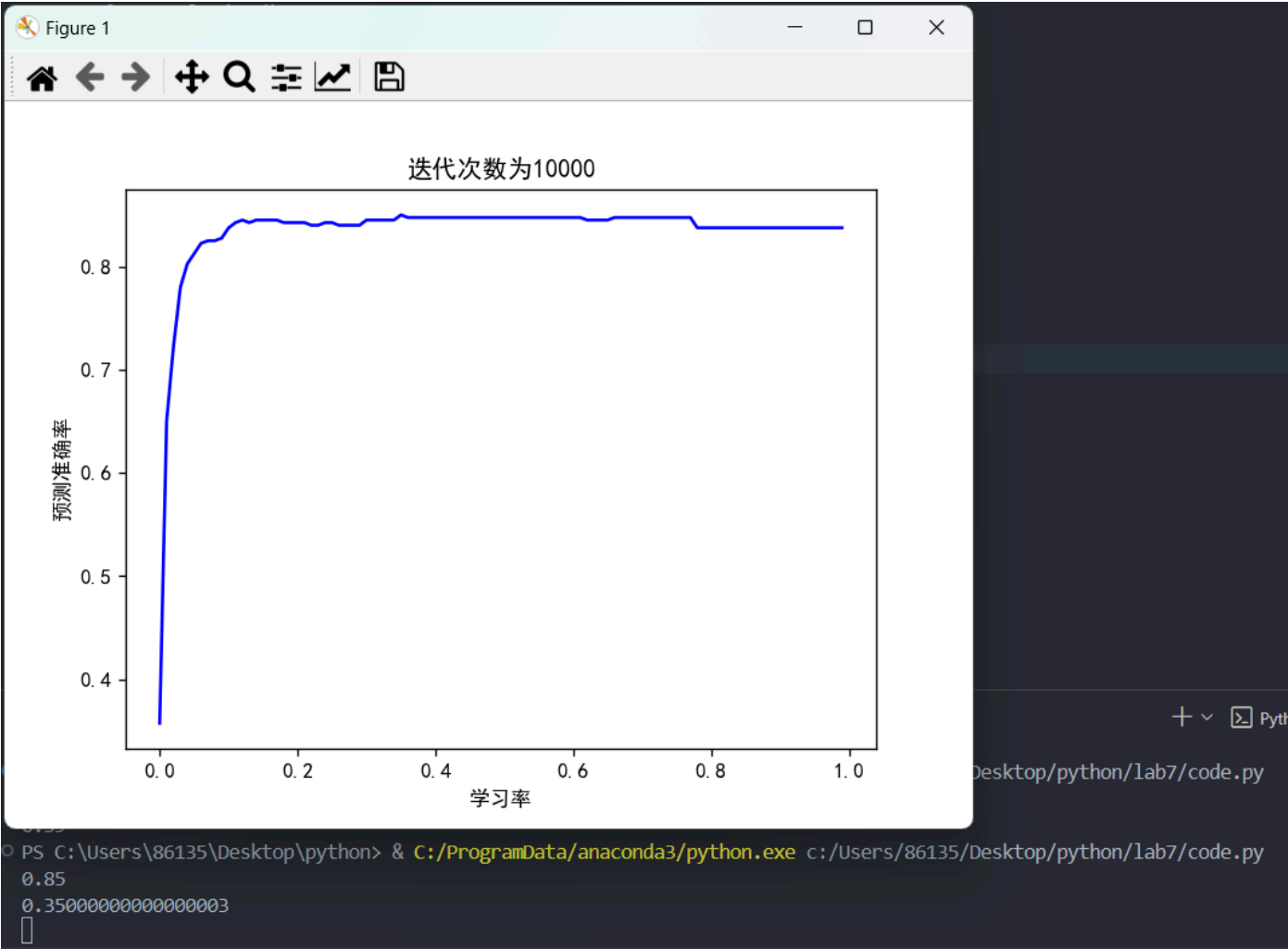


下面的打印参数表示学习率在0.21时取得最好准确率为86.5%。接着继续加大迭代次数，会产生如下结果：





通过比较上图我们可以发现，最佳学习率随着迭代次数的增加不断减小。这是合理的，因为迭代次数的增加使得低学习率下的模型可以有足够的迭代收敛到更好的结果。但是不论怎么改变迭代次数，预测的最优准确率永远保持定值。这时候考虑训练集对结果的影响，改变了训练集后，结果如下：



可以看到最好的预测准确率是和训练集挂钩的。由此为了保证性能，在训练集规定好为前140个数据集的前提下，取较少的迭代次数，经过运行得知，迭代次数为5000时，最优学习率为0.46。

2.评测指标展示及分析（机器学习实验必须有此项，其它可分析运行时间等）

评测指标为准确率和召回率。

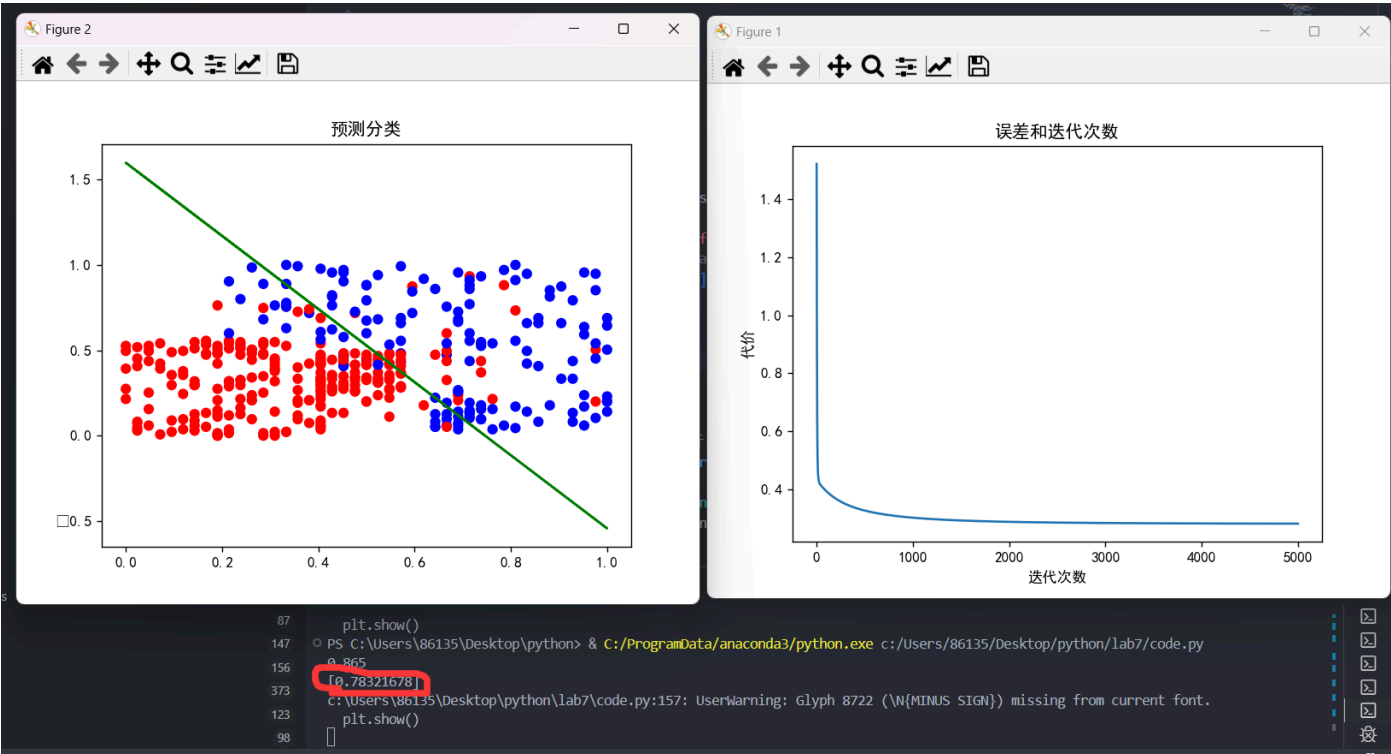
由前面的分析可知，参数调整所能达到的最优准确率是受限于训练集的，由于此处固定采用的前140个数据作为训练集，在我所采用的线性模型下能达到最好值为86.5%，准确率可以接受。

这里假设购房是我们关心的类别，因此正类别是购房，召回率表达式如下：

$$R = \frac{TP}{T}$$

其中TP代表真正类别的样本被正确地预测为正类别的数量，T代表真正类别的样本中正类别的数量。

在训练集固定的情况下，召回率为78.32%:



召回率接近百分之八十，是可以接受的。

综合来看，该模型的预测准确率是86.5%，召回率是78.32%,鉴于训练集为140项数据，训练集数量不算大的情况下，能够达到该准确率是比较不错的。